

GE静态shape多流并行增强技术

作者：刘昌文、熊澜

时间：2026/08

目录

01 多流调度的背景、现状和挑战

模型演进中的性能瓶颈与全图高效并行挑战

02 昇腾亲和的全图多流调度

从并行空间识别到物理流生成，并验证基础多流收益

03 加权控核的多流增强

用真实时间权重与硬件资源约束提升有效并行

04 多流能力的使能与维测

从能力开启到效果确认、问题定位的完整使用闭环

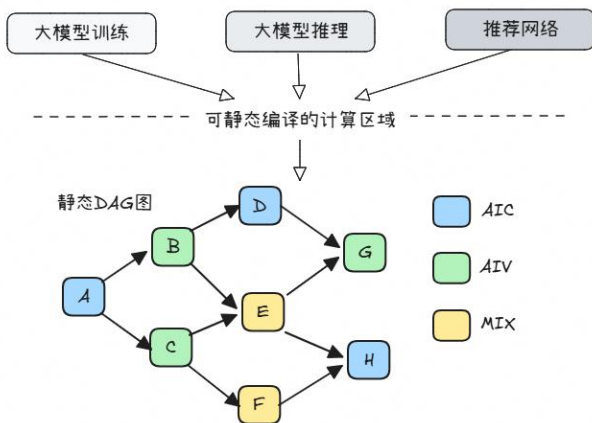
05 Pass接口化与能力扩展

统一多框架接入边界，并探索动态场景

单流串行执行限制硬件并行能力

➤ 主流模型存在大量静态计算区域：

- 大模型训练、推理及推荐网络包含大量结构稳定、重复执行的计算模块
- 固定Batch、固定序列长度或Shape分档时，模型或部分计算区域可形成静态DAG图



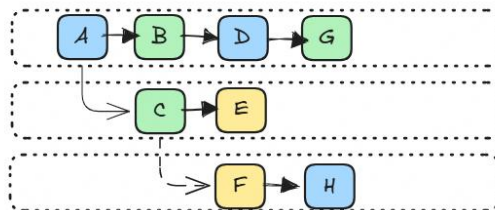
➤ 静态DAG下沉后按单Stream执行：

- GE在编译期完成图优化、内存编排、Tiling，静态DAG整图下沉至Device
- 图中任务进入单一Stream队列，可并行的无依赖算子按照入流顺序执行

当前单流执行图



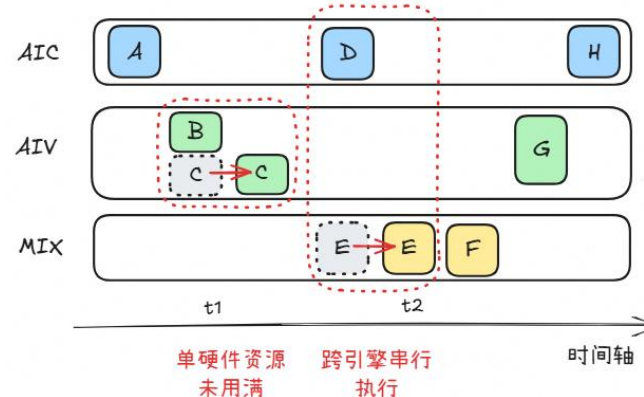
期待并行执行



➤ 两类并行机会未被利用：

- **跨执行引擎**：不同硬件单元本可同时工作，却只能轮流执行
- **同执行引擎**：资源容量本可容纳多个算子，却仍然只能排队

单流执行产生硬件资源浪费



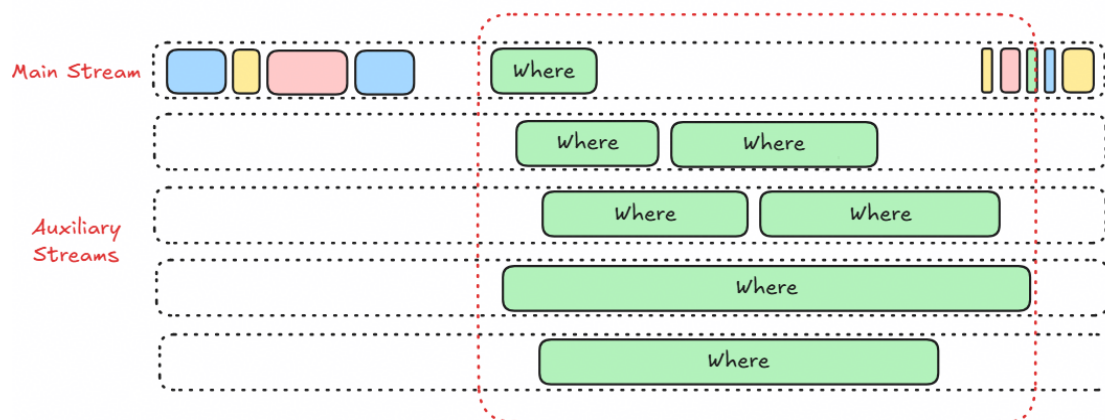
TensorRT辅助多流与昇腾C/V双流并行

GPU侧实践：TensorRT面向可并行Layer的辅助流机制

技术机制：

- 识别网络中具备重叠条件的Layer或独立分支
- 以Main Stream为主，使用Auxiliary Streams并行执行部分Layer

局部多流展开



TensorRT的局部子图多流展开技术

- 已有价值：**局部Layer重叠，减少串行等待。
- 进一步空间：**从部分Layer走向全算子统一排布。

昇腾侧实践：面向Cube/Vector资源的C/V双流并行

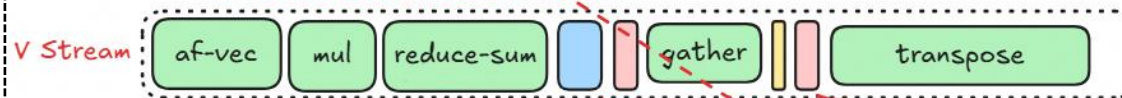
技术机制：

- 按执行资源将任务划分为C类、V类及其他算子
- 分别映射到C Stream、V Stream或沿前驱所在Stream执行

① AIC算子进入C Stream



② 可并行的AIV算子进入V Stream



③ 其他算子沿其前驱所在Stream执行

DeepSeekV4披露的CV控核双流并行技术

- 已有价值：**Cube/Vector Core同时工作，提升资源利用率。
- 进一步空间：**从固定C/V维度走向多类型算子并行。

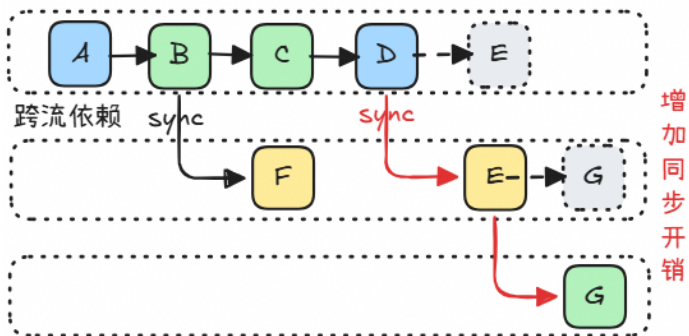
多流调度正在从局部并行、固定双流 → 走向全算子、资源感知的统一调度

从局部并行走向全图高效多流

❑ 多流不是越多越快：同步开销会抵消收益，无效排布不会增加并行，错误映射甚至会造成循环死锁。

➤ 同步越多，收益越小

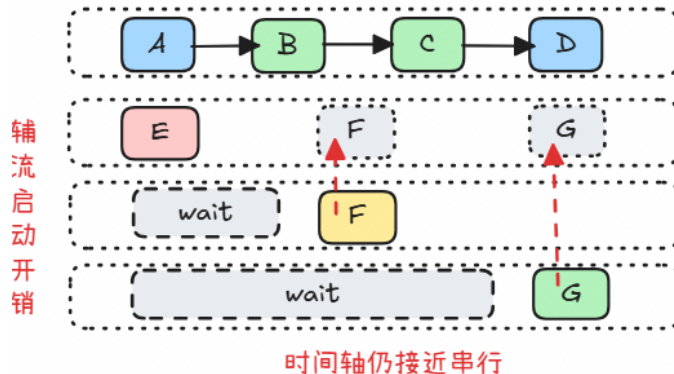
- 跨流需要Event Record/Wait显式同步
- 拆流过细会增加同步点和Event管理开销
- 等待切碎连续执行，压缩有效重叠时间



并行收益被同步等待抵消

➤ 流变多，并行未变

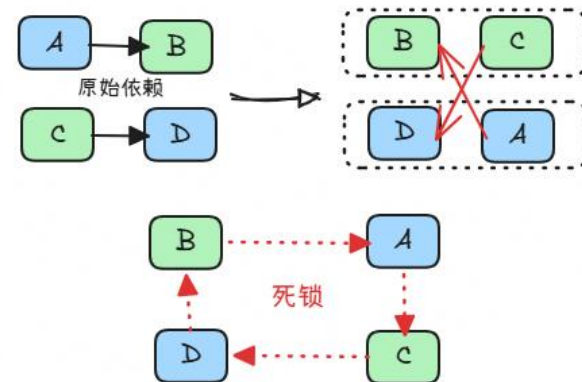
- 图本身并行空间有限，主链过长
- 任务排布失衡，辅流长期空闲或任务扎堆
- 硬件饱和，逻辑并发无法转化为真实重叠



Stream数量 ≠ 有效并行度

➤ 顺序成环，执行停滞

- 原始DAG依赖必须完整保持
- 任务映射到Stream后引入流内固定顺序
- 双依赖叠加，可能错误映射形成等待闭环



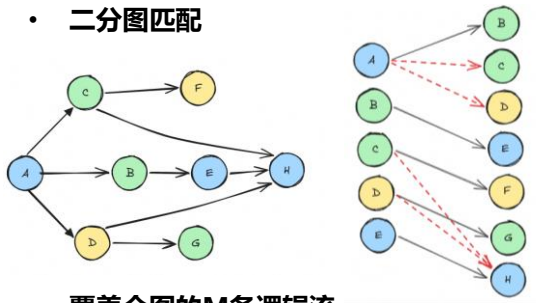
循环等待，执行图停止推进

依赖正确、执行图无环的多流净收益 = 任务重叠收益 - 同步开销 - 流启动开销

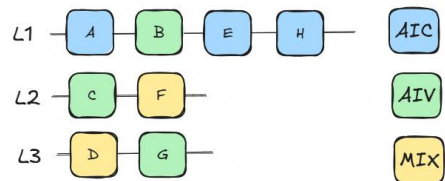
四步生成高效多流执行方案：找出满足约束的并行空间 → 生成物理流 → 细粒度编排 → Device实测选优

① 二分图匹配驱动的逻辑流拆解

• 二分图匹配



• 覆盖全图的M条逻辑流



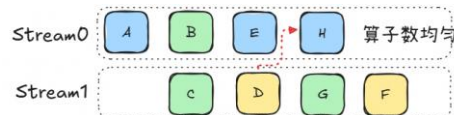
输出：M（覆盖全部节点所需逻辑流）

② 三类合理流合并

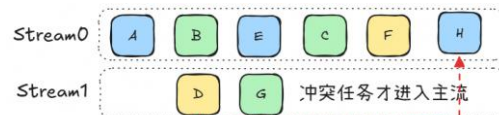
• 用户指定：显式分流单独保留



• 负载均衡：各流任务量接近



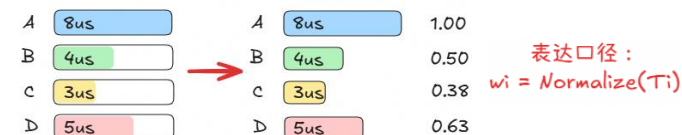
• 主流优先：优先填充主流，必要时新增辅流



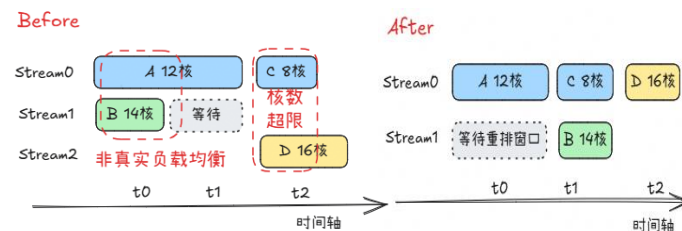
输出：Stream id/流内序/Event边

③ 加权控核增强

• Profiling → 归一化权重



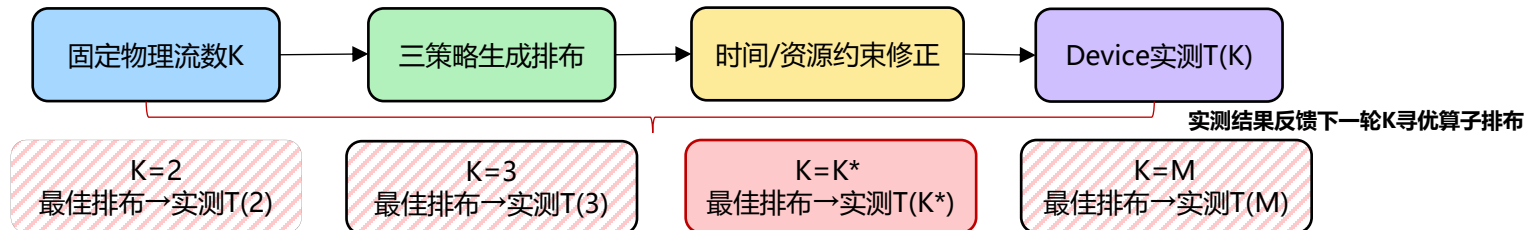
• 资源超限(>22) → 重排或合并



输出：资源合法的Layout候选

④ 固定K内优化，跨K实测选优

- 对每个候选流数K，先由步骤2和3得到该K下最佳Layout，再上Device实测T(K)



最终输出：K* + Strategy* + Layout* → Device实测最优物理流执行图

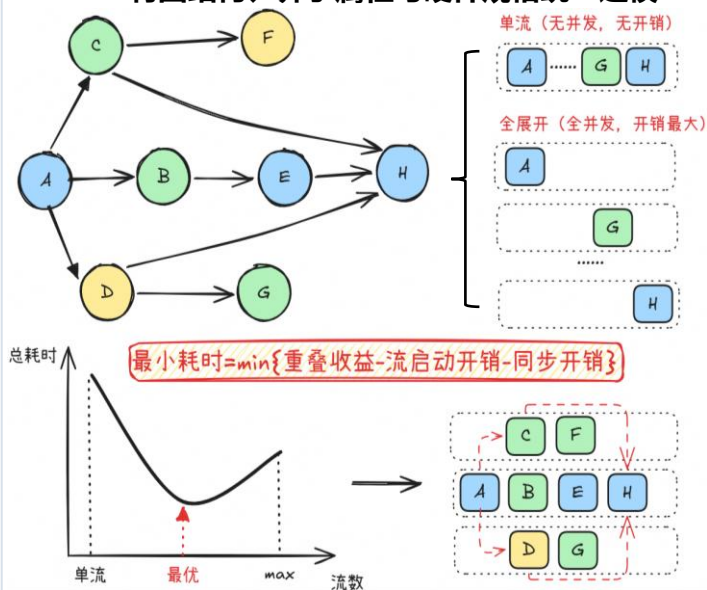
方案闭环：逻辑流拆解定义并行空间 · 三类策略生成物理流 · 加权控核消除资源无效并发 · Device实测闭环选出端到端最优方案

从DAG匹配到多流必要性判断

□ 将DAG拓扑、算子属性与当前硬件规格统一建模，先求出覆盖全图的M条逻辑流，再判断这些逻辑流能否转化为值得保留的有效并发。

➤ 输入建模

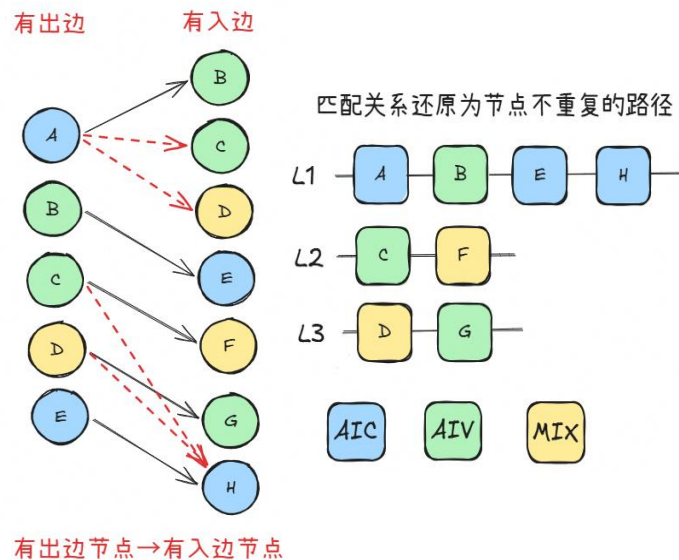
- 将图结构、算子属性与硬件规格统一建模



输出：带硬件属性与约束的DAG

➤ 逻辑流拆解

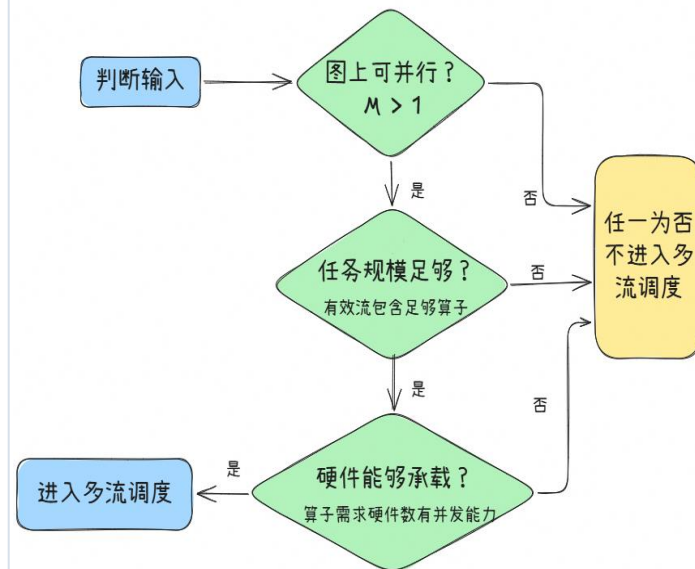
- 最大匹配得到覆盖全图所需逻辑流数量M



输出：M = |节点数| - |匹配边| 条逻辑流

➤ 并发价值判断

- 判断逻辑可拆能否转化为有效并发



输出：逻辑流集合L、数量M

逻辑可拆只是起点：只有拓扑上可并行、任务规模足够且当前硬件能够承载，才进入后续物理流合并与实测寻优

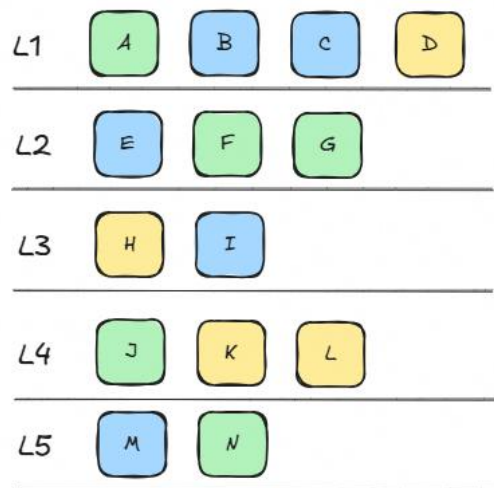
从M条逻辑流到固定K条合理物理流

□ 给定目标物理流数K，使用用户指定、负载均衡和主流优先三类策略生成候选，再综合Event代价、有效重叠和执行正确性，输出当前K下的合理物理流执行图。

➤ 输入

• M条覆盖全图的逻辑流

本轮固定目标：K=3条物理流



输入：逻辑流集合L+目标物理流数K

➤ 候选生成

• 三类合并策略：M条逻辑流 → K条物理流

> 用户指定

显式流归属优先



> 负载均衡

各流算子数/任务量接近



> 主流优先

先填主流空窗，冲突再分流



➤ 约束筛选

• 候选需满足的执行要求

> Event插入开销(成本)

只做必要的依赖插入Record/Wait，减少不必要同步

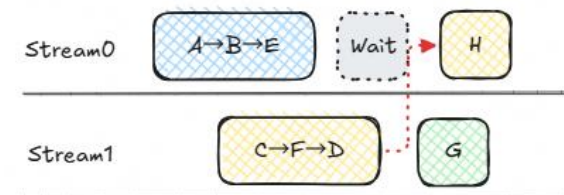
> 有效并行(收益)

物理流之间要形成真实重叠，避免长期等待伪并行

> 正确并行(底线)

依赖保持、Event完整、执行图无环

> 固定K下合理候选：

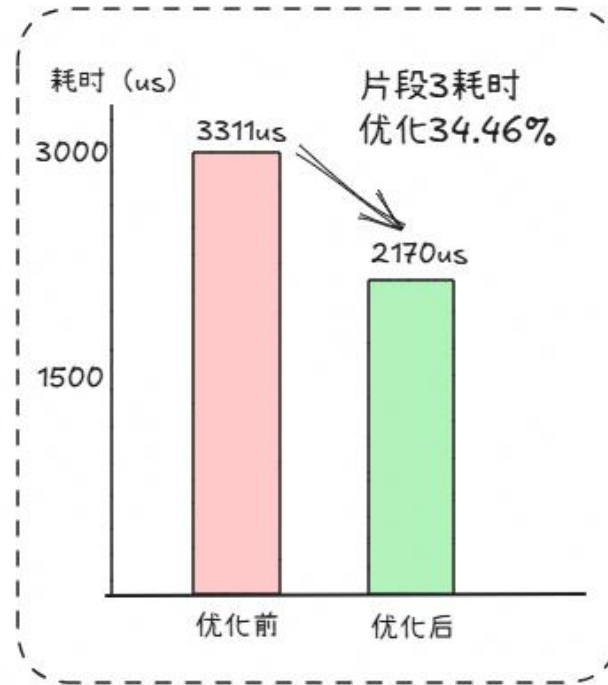
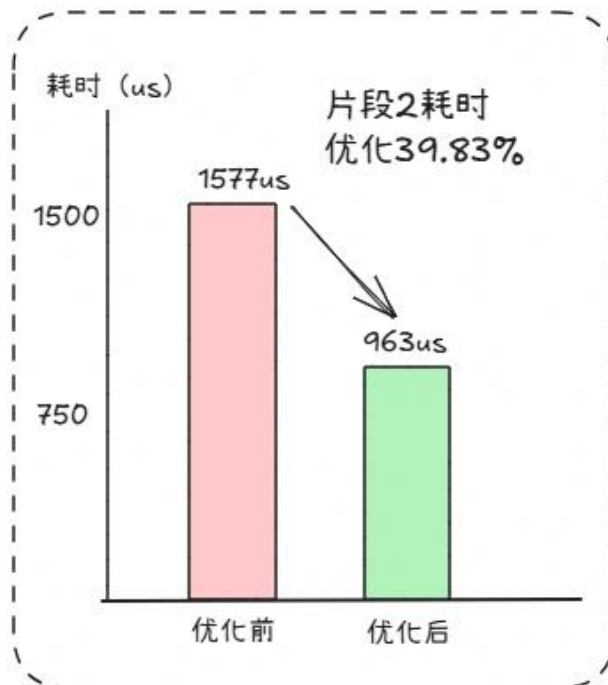
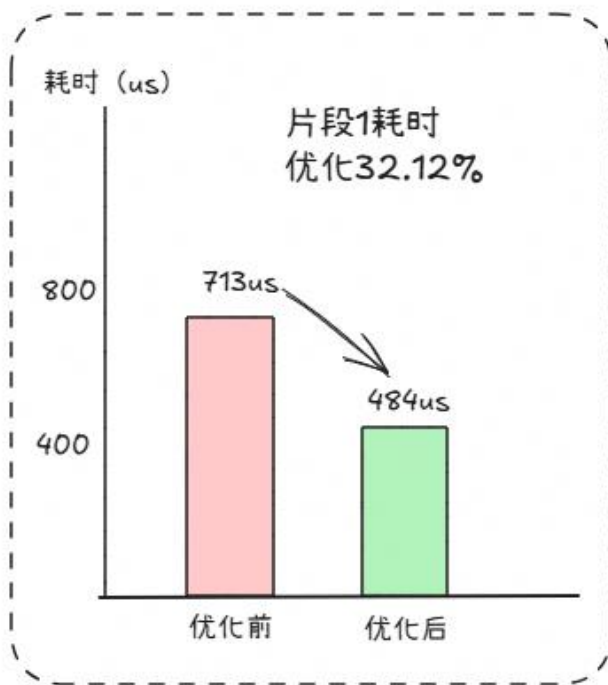


输出：固定K下的最优物理流编排

三类策略负责生成候选，执行要求负责筛选；目标是在依赖正确的前提下，用更少Event换取更多真实重叠

无历史Profiling条件下的多Case实测

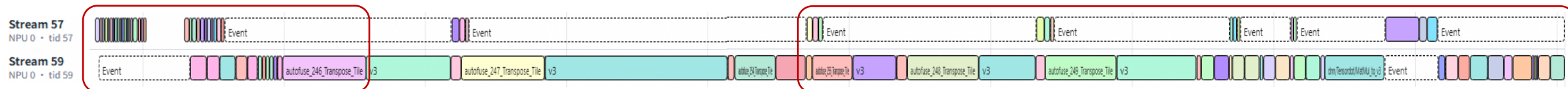
□ 首次执行：无历史Profiling输入，仅启用逻辑流拆解与物理流合并



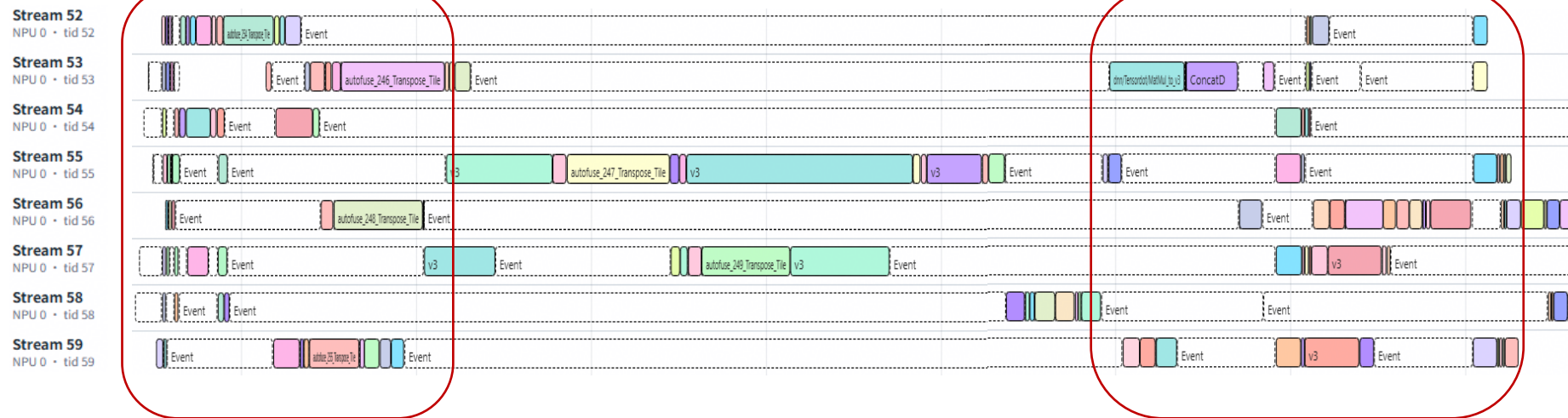
无历史Profiling输入时，已能在具备图并行空间的Case中获得端到端收益

典型Case的单流与多流Timeline对比

□ 同一Case、同一硬件、同一输入、同一时间尺度；Profiling仅用于观测，不参与本轮调度决策



- 并行执行度显著提升
- 缩短关键路径



多流优化前

端到端收益20%+

多流优化后

多流没有减少算子本身计算量，而是将原本串行排队的独立任务转化为真实执行重叠，从而缩短关键路径

从Profiling权重到资源合法排布

□ 用首轮Profiling获得算子真实时间权重，再结合AIC/AIV资源上限修正排布：超限任务重排，无效并发流合并，得到硬件真正能够承载的多流图。

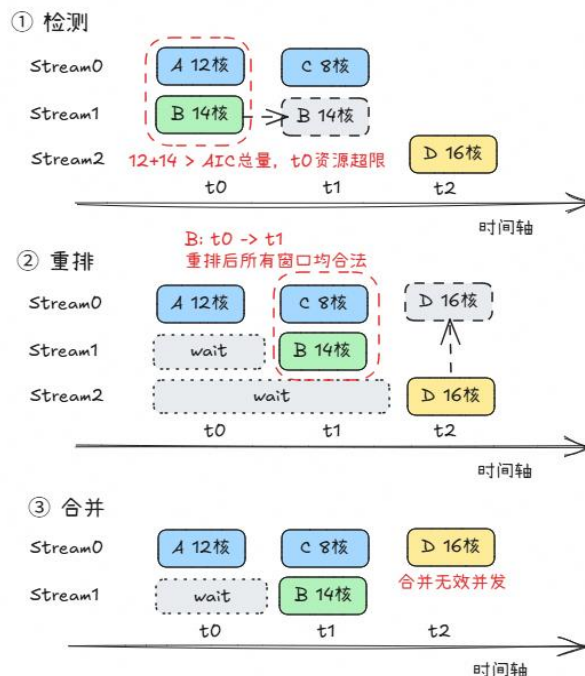
➤ Profiling归一化



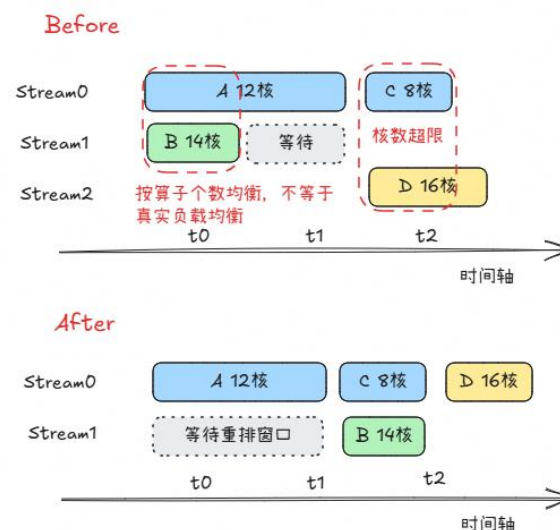
表达口径： $w_i = \text{Normalize}(T_i)$

输出：带时间权重的候选多流图

➤ 资源超限：重排或合并



➤ 资源合法的新多流结果

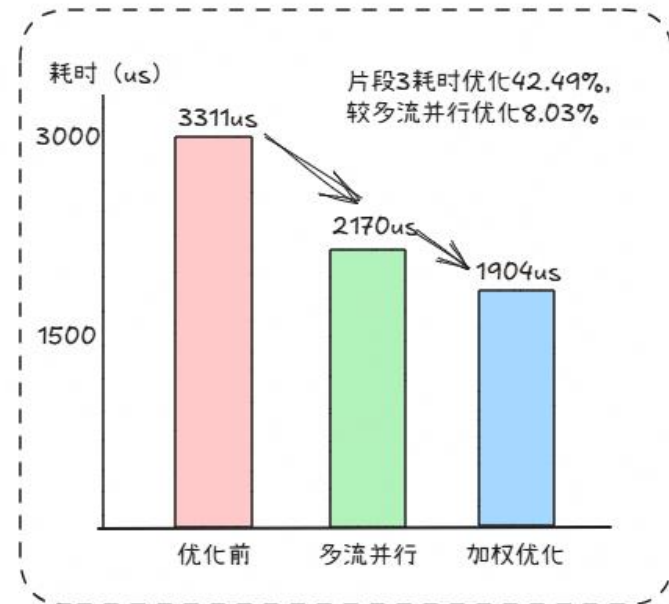
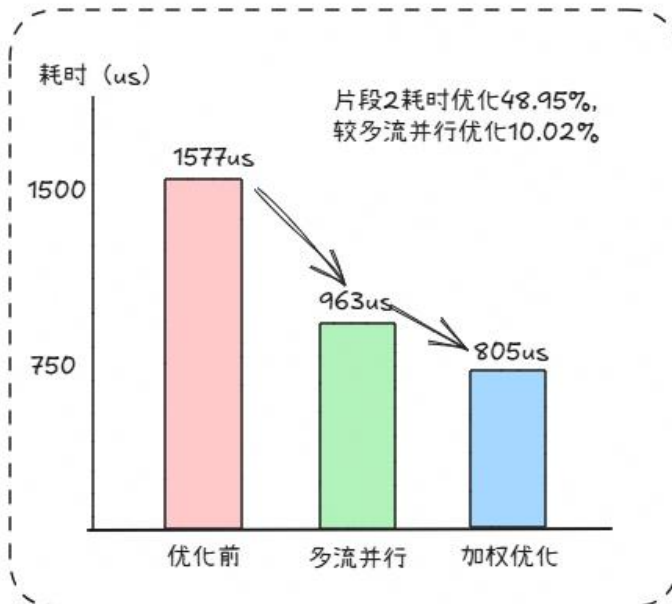
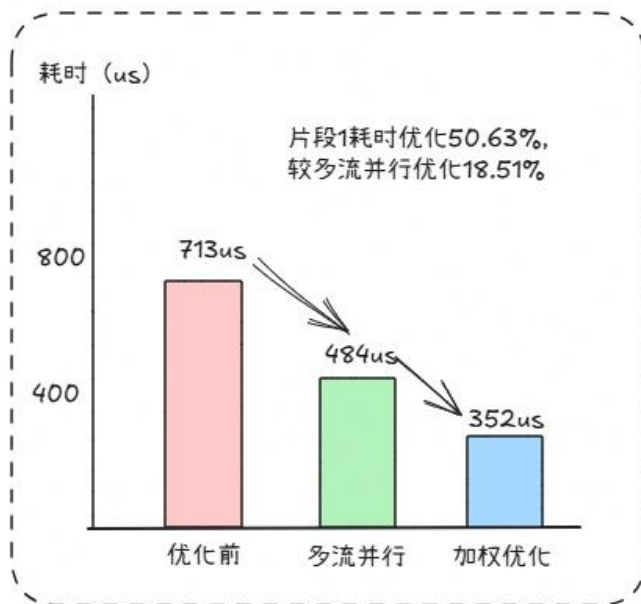


输出：有效重叠更充分的物理流图

用真实时间衡量负载、用硬件上限约束并发，把逻辑多流修正为资源合法且有效重叠的物理多流

加入步骤三后的多Case实测

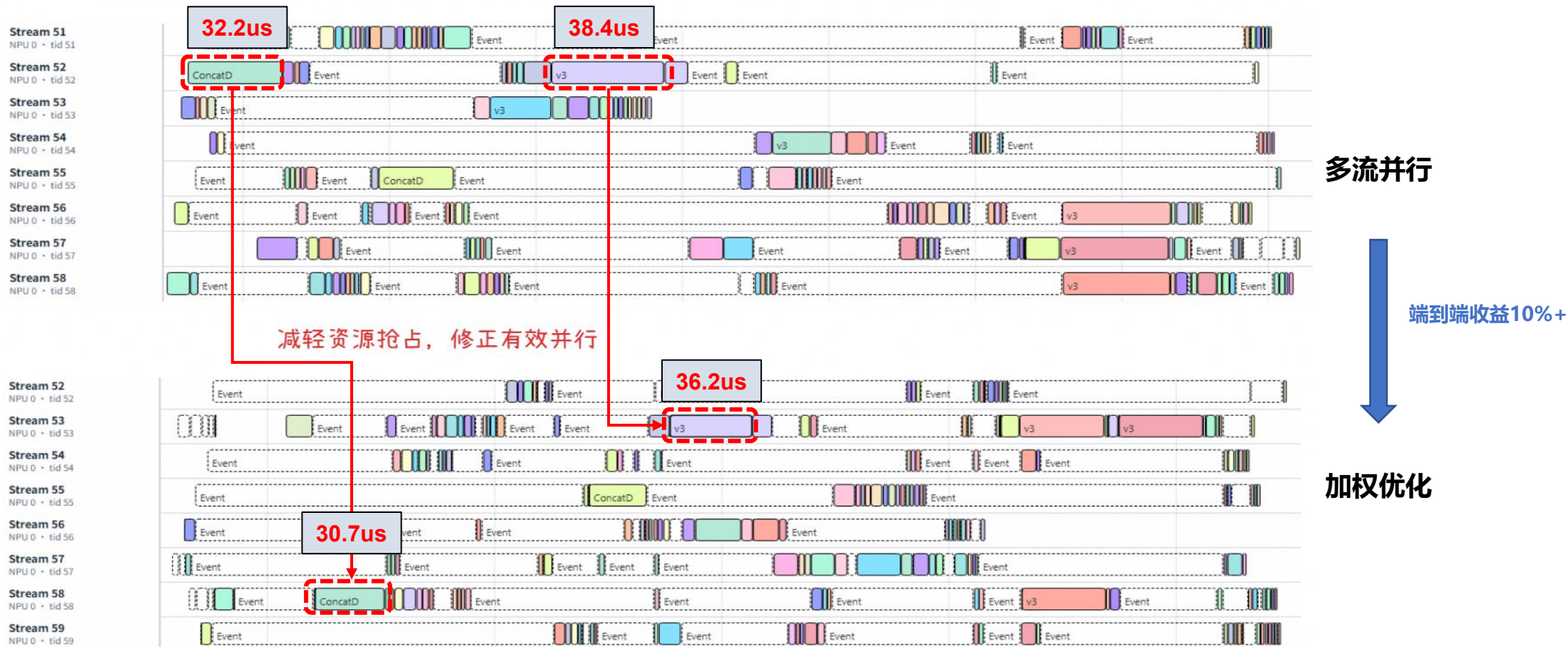
□对比基础多流与加权控核后的端到端表现，验证真实时间权重和硬件资源约束带来的增量收益



通过真实时间权重修正负载、硬件资源约束消除冲突与无效并发

典型Case的步骤三前后Timeline对比

□ 在同一Case和同一时间尺度下，对比步骤三前后的Stream与AIC/AIV资源Timeline，定位关键路径缩短的具体来源



时间加权修正负载，资源约束消除冲突：留下的不是更多Stream，而是更有效的硬件并行

从多流使能到效果维测

功能开关

使能方式 1

```
# tensorflow 入口
import tensorflow as tf
NPU_config = tf.ConfigProto()
custom_op =
NPU_config.graph_options.rewrite_
    options.custom_optimizers.add()
custom_op.parameter_map["auto_multistream_parallel_mode"].s =
tf.compat.as_bytes("LoadBalance:8")
```

使能方式 2

```
# ge session 入口
std::map<ge::AscendString,
ge::AscendString> sess_init_options;
sess_init_options[ge::AscendString("ge.auto
MultistreamParallelMode")] =
ge::AscendString("MainStream:20");
```

当前社区版本已支持TorchAir和ATC入口，这里不再赘述

参数配置

参数格式:

Strategy : StreamNums

> strategy可选: LoadBalance, MainStream, WeightedLoadBalance, 分别为负载均衡、主流、加权均衡三种多流模式

> StreamNums: 划分的流数，不超过64

维测开关

使能

```
export ASCEND_GLOBAL_LOG_LEVEL=1
export ASCEND_SLOG_PRINT_TO_STDOUT=1
```

Info log

> 在log信息中，搜索dag_stream_allocator_pass.cc dag_stream_allocator.cc dag_stream_merger.cc dag_weighted_stream_merger.cc，即可看到所有与GE多流相关的log

profiling

```
PROF_XXXX_XXXX_XXXX
device_0
host
mindstudio_profiler_log
mindstudio_profiler_output
api_statistic_xxx.csv
fusion_op_xxx.csv
msprof_xxx.json
op_statistic_xxx.csv
op_summary_xxx.csv
README.txt
step_trace_xxx.json
step_trace_xxx.csv
task_time_xxx.csv
```

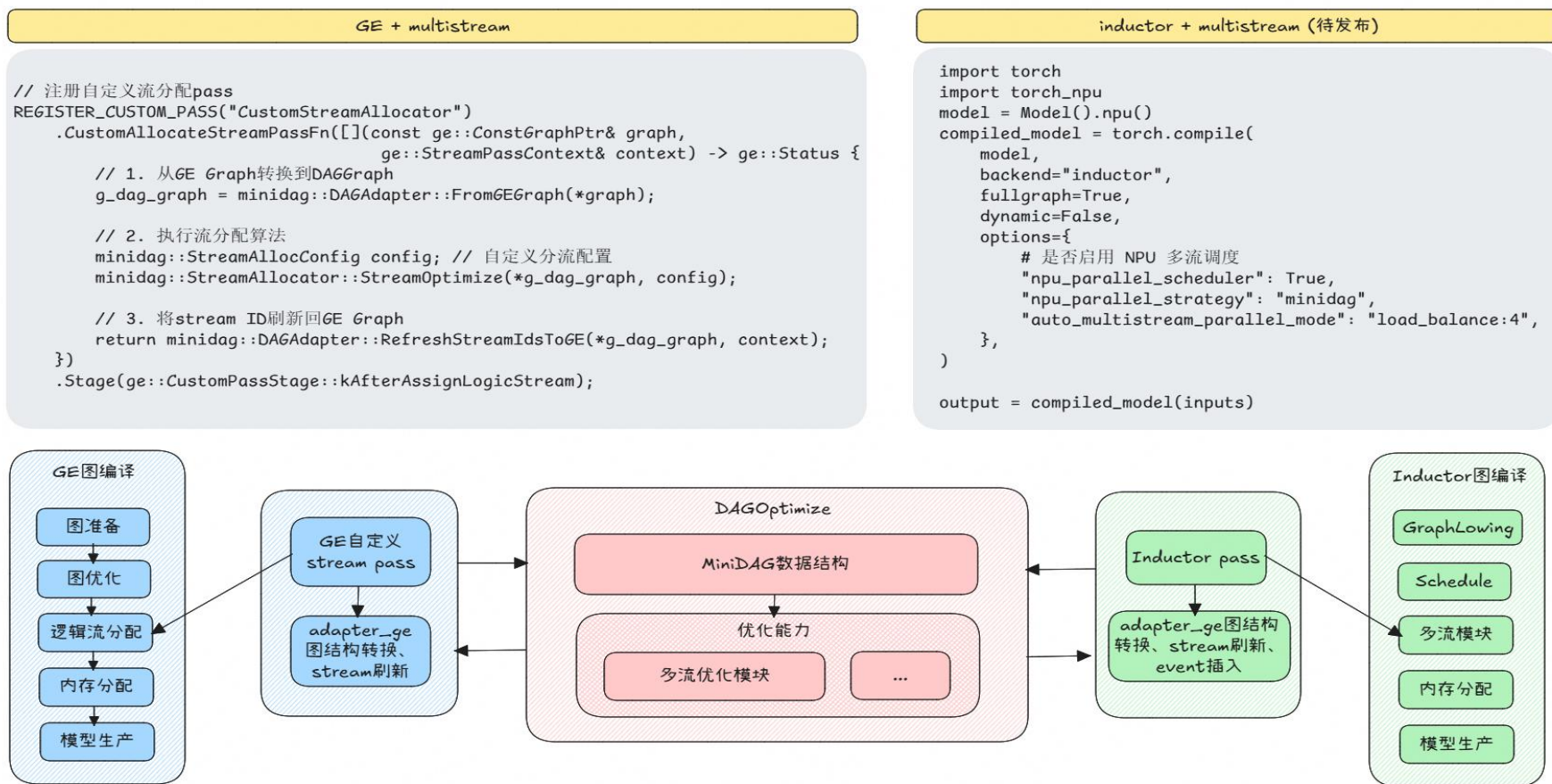
> msprof --output=path --application="执行命令"，即可在指定路径下输出profiling

> 通过profiling查看工具打开msprof_xxx.json，即可看到算子流分布情况

产物

Pass接口化统一复用多流能力

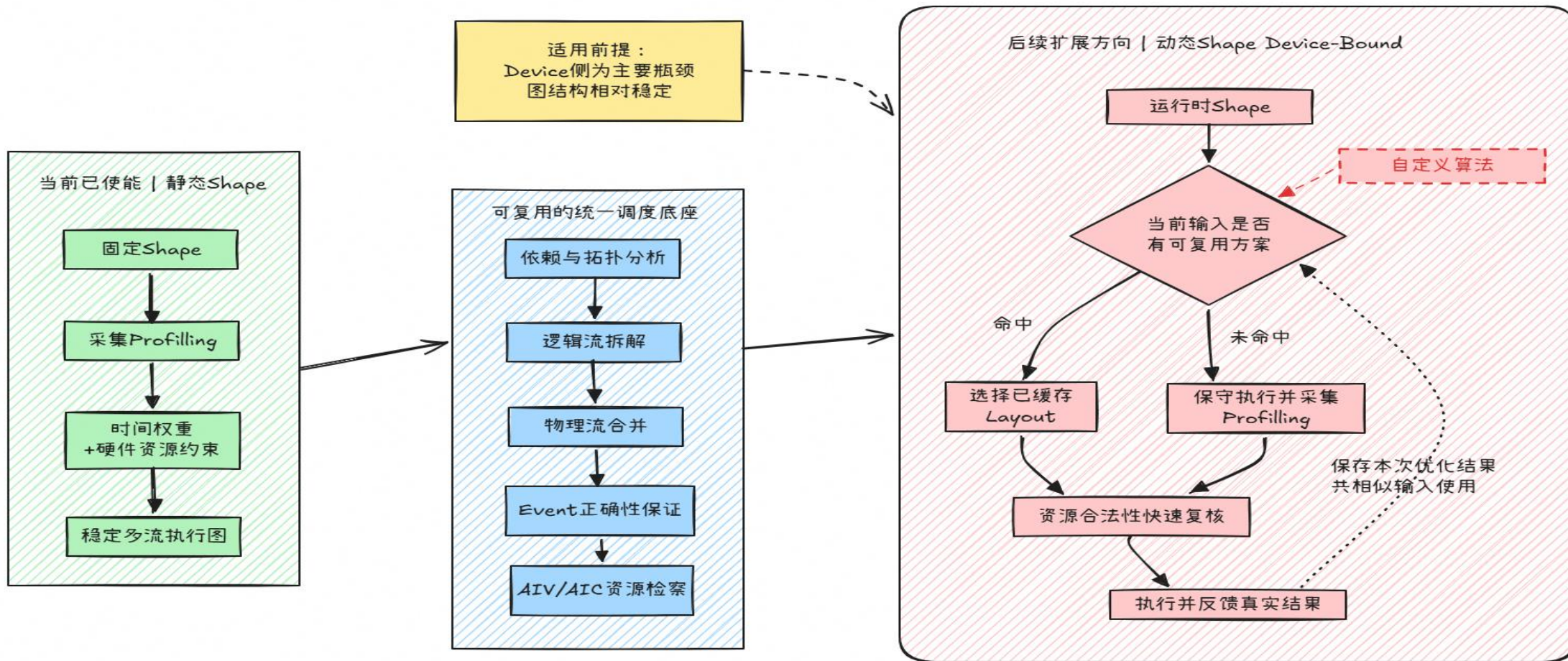
□ 各框架通过外部Pass接口明确输入输出和调用边界，各框架可沿各自编译链路完成接入，并复用多流优化组件归一的多流调度能力



Pass接口化把框架差异收敛在接入层，将图分析、流合并与资源调度沉淀为可复用的GE能力

从静态Shape到动态Shape Device-Bound场景

□ 面向动态Shape，可复用同一**调度框架**或**自定义算法**，通过Shape分档、运行时Profiling和候选切换适配不断变化的真实负载



当动态Shape场景的主要瓶颈位于Device侧且Shape变化具有可复用规律时，这套能力仍有进一步优化空间

直播有奖竞答

直播有奖互动（评论答复选择题答案即可参加）：

奖品：CANN定制鼠标垫，10份

<https://gitcode.com/cann/ge/discussions/8>

直播作业：寻找GE仓example示例里的最优多流配置

奖品：Huawei FreeBuds6耳机，2份

<https://gitcode.com/cann/ge/issues/516>



有奖直播互动



直播有奖作业

<https://gitcode.com/cann>

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯